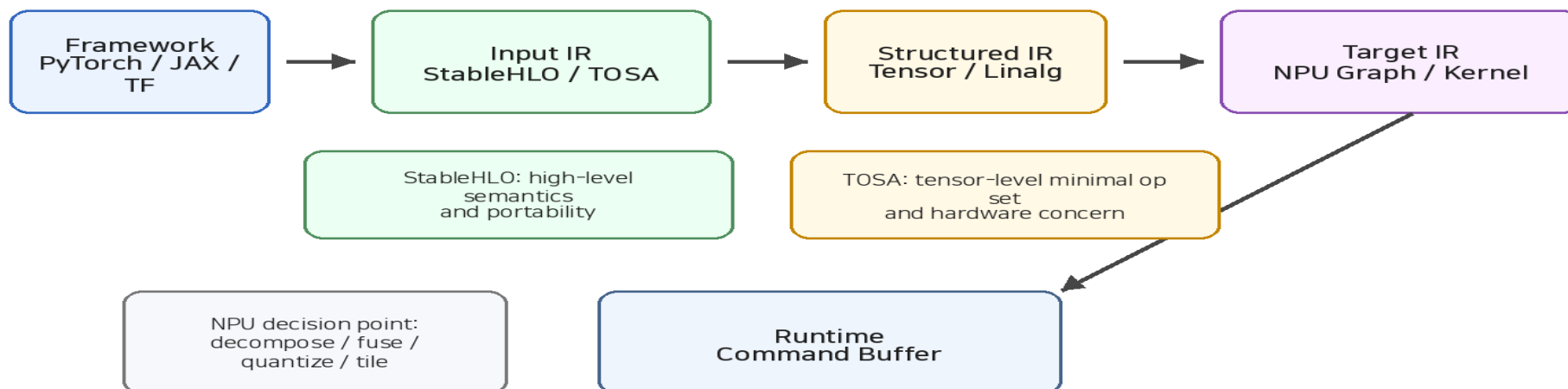


MLIR NPU Compiler - Lecture 06

StableHLO, TOSA, and AI Model Input IR

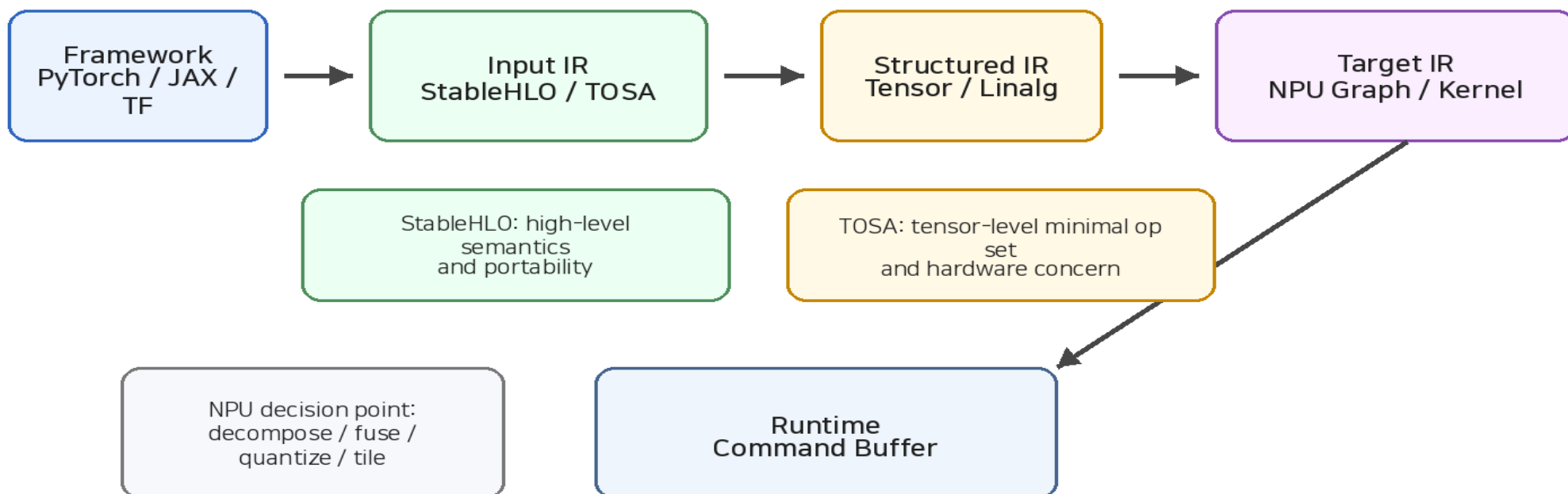
Lecture 06: StableHLO / TOSA as AI Model Input IR



오늘의 위치

5 강 Dialect 시스템 이후, 실제 AI model input IR 로 들어간다

Lecture 06: StableHLO / TOSA as AI Model Input IR



Learning Objectives

StableHLO 와 TOSA 가 NPU compiler input IR 로 쓰이는 이유를 설명한다 .
StableHLO/TOSA 의 abstraction level, op semantics, legalization boundary 를 비교한다 .
LayerNorm 과 GELU 를 primitive operation graph 로 분해한다 .
NPU legal op set 기준으로 unsupported op, fallback, fusion candidate 를 audit 한다 .
MatMul + Bias + GELU, LayerNorm subgraph 의 decomposition/fusion policy 를 설계한다 .

왜 Input IR 이 중요한가 ?

Bad / Risk

Framework graph 를 바로 target kernel 로 매핑
Composite op 를 무조건 제거
Shape/layout/quantization audit 없이 lowering
Numerical approximation 을 test 없이 변경

Good / Recommended

Input IR 을 semantic contract 로 사용
보존할 pattern 과 분해할 pattern 을 구분
Legal op matrix 를 먼저 작성
Approximation 과 quantization policy 를
verifier/test 에 연결

Model Export Path

PyTorch / JAX / TensorFlow -> StableHLO or TOSA -> canonicalize -> decompose -> NPU
graph/kernel

Importer 는 original op name/location 을 보존한다 .

Static/dynamic dimension 정보를 audit 한다 .

Layout intent 와 quantization metadata 를 잃지 않는다 .

Backend 는 legal op list 와 비교하고 unsupported op 는 decompose 또는 fallback 처리한다 .

StableHLO: high-level semantics

ML model high-level operations 를 표현하는
operation set

ML framework 와 ML compiler 사이 portability
layer

NPU compiler 의 importer boundary 로 적합
dot_general, convolution, reduce, broadcast,
reshape, elementwise op 가 핵심

```
stablehlo.dot_general  
stablehlo.convolution  
stablehlo.reduce  
stablehlo.broadcast_in_dim  
stablehlo.reshape  
stablehlo.transpose  
stablehlo.erf / tanh  
stablehlo.add / multiply
```

StableHLO 를 NPU 관점으로 읽기

dot_general: MatMul, batched MatMul, attention projection mapping 후보
convolution: layout rewrite, Winograd/direct convolution, SRAM tile policy 후보
reduce + broadcast_in_dim: LayerNorm/Softmax/RMSNorm pattern 의 핵심 signal
erf/tanh/sigmoid: activation approximation 과 LUT/table 선택의 신호
dynamic shape op: edge NPU 에서는 specialization, guard, fallback decision 필요

TOSA: tensor-level operator set

MLIR TOSA dialect 는 TOSA specification 을
구현

tensor-level abstraction 에서 operator set 을
정의

CPU/GPU/NPU target concern 을 고려한
minimality 철학

target legal op audit 과 quantized path 분석에
유리

```
tosa.matmul  
tosa.conv2d  
tosa.reduce_sum  
tosa.rsqrt  
tosa.add / sub / mul  
tosa.reshape / tile  
tosa.rescale  
tosa.table
```


TOSA 를 NPU 관점으로 읽기

matmul/conv2d: target kernel mapping 이 명확한 core compute op
reduce_sum/rsqrt: LayerNorm, RMSNorm, Softmax lowering 의 primitive
rescale: quantized accumulator -> output dtype requantization path
table: INT8/INT4 activation LUT 또는 nonlinear approximation 후보
minimal op set: backend fusion 은 compiler 가 담당해야 한다는 신호

StableHLO vs TOSA: Compiler 관점 비교

관점	StableHLO	TOSA
Role	Portability layer	Tensor-level op set
Level	High-level HLO	Backend-friendly tensor
Good for	Semantic preservation	Legal op audit
Risk	Late decomposition	Lost fusion intent
NPU focus	dot_general/reduce/broadcast	matmul/reduce_sum/rescale/table

Composite op vs Primitive op

LayerNorm/GELU/Softmax 는 model layer 로는 하나의 의미지만 IR 에서는 여러 primitive op pattern 으로 나타난다 .

Composite intent 를 보존하면 fusion 과 numerical policy 를 관리하기 쉽다 .

Primitive 로 분해하면 target legal op set, scheduling, bufferization 을 진행하기 쉽다 .

NPU compiler 는 보존 / 분해 /fuse/fallback 의 경계를 pass pipeline 으로 명시해야 한다 .

LayerNorm math

```
x: [B, S, K], gamma: [K], beta: [K]
```

```
mean = reduce_sum(x, axis=K) / K
```

```
center = x - broadcast(mean)
```

```
var = reduce_sum(center * center, axis=K) / K
```

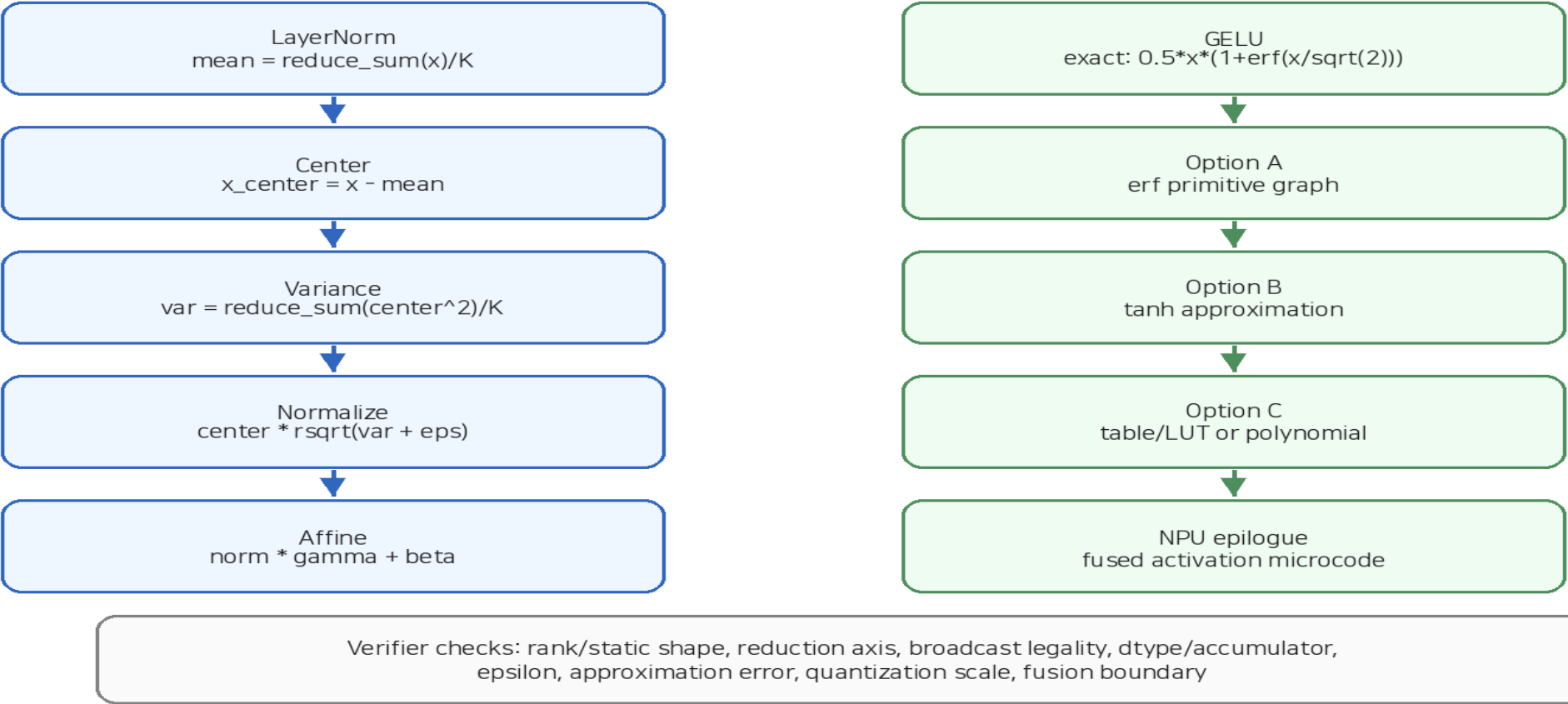
```
inv_std = rsqrt(var + epsilon)
```

```
norm = center * broadcast(inv_std)
```

```
out = norm * gamma + beta
```

LayerNorm primitive graph

Primitive Decomposition for NPU Legalization



StableHLO LayerNorm pattern signal

```
stablehlo.reduce(axis=K) -> mean  
stablehlo.broadcast_in_dim(mean)  
stablehlo.subtract(x, mean_b)  
stablehlo.multiply(center, center)  
stablehlo.reduce(axis=K) -> variance  
stablehlo.add(variance, epsilon)  
stablehlo.rsqrt
```

stablehlo.multiply + stablehlo.add for gamma/beta

TOSA LayerNorm pattern signal

```
tosa.reduce_sum(axis=K)
tosa.tile or broadcast materialization
tosa.sub / tosa.mul
tosa.reduce_sum(axis=K)
tosa.add(epsilon)
tosa.rsqrt
tosa.mul + tosa.add
```

NPU audit: is reduce_sum axis legal? is rsqrt supported? is tile materialized?

GELU math

Exact:

$$\text{GELU}(x) = 0.5 * x * (1 + \text{erf}(x / \sqrt{2}))$$

Tanh approximation:

$$\text{GELU}(x) \approx 0.5 * x * (1 + \tanh(\sqrt{2/\pi}) * (x + 0.044715 * x^3)))$$

GELU lowering options for NPU

Bad / Risk

erf exact graph 를 보존
special function kernel 또는 CPU fallback
golden output 과 비교 쉬움

Good / Recommended

tanh approximation
sigmoid/polynomial approximation
INT8/INT4 table/LUT
MatMul epilogue fused activation

LayerNorm/GELU fusion opportunities

MatMul + Bias + GELU: accumulator -> bias add -> activation -> optional requant 를 epilogue 로 fuse 가능

LayerNorm: reduction + elementwise chain 을 row/tile 단위로 fuse 하여 DRAM traffic 감소

Broadcast gamma/beta 는 materialization 하지 않고 vector load 또는 SRAM resident constant 로 처리

Fusion 이 항상 좋은 것은 아님 : producer/consumer reuse, SRAM pressure, quantization boundary 를 함께 봐야 함

NPU Legal Op Audit Matrix

NPU Legal Op Audit Matrix

Op	StableHLO	TOSA	NPU support	Action	Test
MatMul	dot_general	matmul	MXU/Systolic	legalize to npu.matmul	shape + K alignment
LayerNorm	reduce/add/rsqrt	reduce_sum/rsqrt	partial	decompose + fuse	epsilon + axis
GELU	erf/tanh graph	tanh/table graph	approx/LUT	select approximation	max error
Softmax	reduce_max/exp/sum	exp/reduce_sum	partial	tile + stable math	overflow test
Conv2D	convolution	conv2d	direct/winograd	layout rewrite	NHWC/HWCF

Use this matrix before writing a lowering pass: unsupported op -> decompose; supported composite -> maybe preserve and fuse.

Shape and Dynamism Policy

Edge NPU 는 대부분 compile-time static tile 과 memory plan 을 선호한다 .
Dynamic batch/sequence 는 specialization, guard, or fallback 으로 policy 를 정해야 한다 .
LayerNorm K axis 가 dynamic 이면 reduction tile 과 vector tail 처리 문제가 커진다 .
StableHLO dynamic op 를 input 에서 받더라도 NPU dialect 로 내려가기 전에는 legality 를
확정해야 한다 .

DType and Quantization Policy

FP16/BF16 path: MatMul accumulator, LayerNorm reduction accumulator precision 을 정의한다 .

INT8/INT4 path: scale/zero-point, per-tensor/per-channel, rescale/requant 위치를 보존한다 .
GELU/Softmax 같은 nonlinear op 는 table/LUT 또는 higher-precision fallback 이 필요할 수 있다 .

TOSA rescale/table style op 는 quantized NPU path 에서 중요한 semantic signal 이다 .

Layout Policy: logical vs physical

StableHLO/TOSA 의 shape 는 logical tensor semantics 다 .

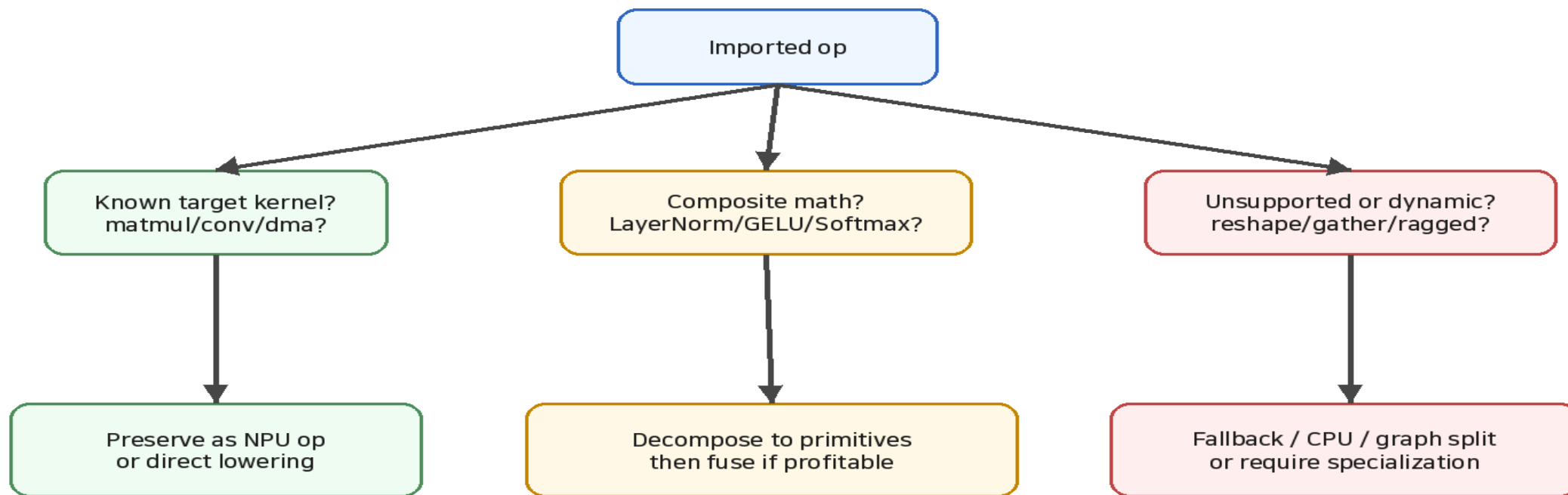
NPU kernel 은 blocked/K-major/NHWC/HWIO 등 physical layout 을 요구할 수 있다 .

broadcast/reshape/transpose 는 실제 data movement 인지 view 인지 반드시 구분한다 .

Layout propagation 은 bufferization 전에 fusion, vectorization, DMA planning 과 함께 결정해야 한다 .

Recommended Legalization Pipeline

Input IR Decision Tree for NPU Compiler



Rule of thumb: StableHLO/TOSA input IR is a semantic contract, not the final hardware schedule.

Example: MatMul + Bias + GELU

```
stablehlo.dot_general  
  -> stablehlo.broadcast_in_dim(bias)  
  -> stablehlo.add  
  -> GELU exact or tanh/table approximation
```

NPU decisions:

1. map dot_general to npu.matmul
2. keep bias in epilogue if possible
3. choose GELU approximation
4. insert requant/rescale if INT8/INT4 path

Example: LayerNorm audit

Input: $x[1, 128, 768]$, $\text{gamma}[768]$, $\text{beta}[768]$

Reduction axis: $K=768$, static and vectorizable?

Accumulator: f32 or widened fixed-point?

Epsilon: compile-time attribute or constant SSA value?

Broadcast gamma/beta: materialize or vector-load?

Output: preserve fp16/bf16 or requantize to int8?

너무 빨리 lower 하지 말아야 할 것

Composite intent: LayerNorm/GELU/Softmax pattern identity

Original framework location: debugging 과 accuracy triage 에 필요

Quantization metadata: scale/zero-point/calibration origin

Approximation policy: exact/approx/LUT 와 error budget

Logical shape/layout: physical materialization 결정 전까지 보존

Testing Strategy

IR regression: pattern 이 의도한 primitive graph 로 canonicalize/decompose 되는지 FileCheck
Numerical golden: LayerNorm/GELU exact vs approximation output error
Shape edge case: aligned K, unaligned K, dynamic B/S, singleton dimensions
Quantized path: rescale/table/LUT saturation 과 rounding policy
Fallback path: unsupported op 가 graph split 또는 CPU fallback 으로 안전하게 처리되는지

Exercise A: Op classification

StableHLO/TOSA 예제 파일에서 op 를 모두 나열한다 .

각 op 를 core compute / reduction / elementwise / reshape-broadcast / quantization / unsupported 로 분류한다 .

NPU target support 여부를 표시한다 .

Preserve / decompose / fuse / fallback 중 하나의 action 을 선택한다 .

Exercise B: LayerNorm primitive graph

Input: $x[B,S,K]$, $\gamma[K]$, $\beta[K]$

Fill the table:

Step	SSA value	Shape	Kernel mapping	Verifier rule
------	-----------	-------	----------------	---------------

Must include:

mean, center, variance, inv_std, γ/β broadcast, output

Exercise C: GELU approximation policy

정확도 우선 모델 : erf exact 또는 tanh approximation?

INT8/INT4 모델 : table/LUT 를 쓸 수 있는 calibration 조건은 ?

MatMul epilogue fusion: bias, activation, requant order 를 어떻게 보존할 것인가 ?

Regression test: max error, mean error, top-k accuracy, saturation case 를 정의한다 .

Homework

MatMul + Bias + GELU subgraph 를 StableHLO 또는 TOSA pseudo-MLIR 로 작성한다 .
LayerNorm decomposed graph 의 shape table 을 완성한다 .
NPU legal op audit matrix 를 작성한다 .
Unsupported op 3 개에 대한 fallback/decomposition plan 을 제안한다 .

Takeaways

StableHLO/TOSA 는 NPU command buffer 가 아니라 input semantic contract 다 .
LayerNorm/GELU 는 input IR legalization, decomposition, fusion policy 를 연습하기 좋은 핵심 예제다 .
NPU compiler 는 op support, shape, dtype, layout, quantization, numerical approximation 을 함께 audit 해야 한다 .
좋은 pass pipeline 은 composite intent 를 보존하면서 target legal primitive 로 내려간다 .

References

StableHLO Specification - <https://openxla.org/stablehlo/spec>

StableHLO Overview - <https://openxla.org/stablehlo>

MLIR TOSA Dialect - <https://mlir.llvm.org/docs/Dialects/TOSA/>

TOSA Specification - https://www.mlplatform.org/tosa/tosa_spec.html

MLIR Passes - <https://mlir.llvm.org/docs/Passes/>